

C++ Programmeermethoden, Assignment4: GPU Programming

Bas Terwijn

May 6, 2026

1 Introduction

In this assignment we write parallel GPU implementations of some common algorithms. We will compare the execution time of the GPU implementations with the CPU implementations. You will have to work on a computer with NVIDIA GPU and CUDA installed, your options include:

- your own computer with NVIDIA GPU if you have one
- Free Google Colab, using these instructions: GPU Programming (I'm not sure this works for large 'n' values)
- Snellius, instructions will be provided as Canvas announcement

1.1 Vector Normalization

Write in file "normalize_vec_gpu.cu" a parallel GPU implementation of the following C++ code that normalizes a vector. First test with low 'n=18' value but make sure it parallelizes for large 'n=1000000' value and compare the execution time of the GPU implementation with the CPU implementation.

```

normalize_vec_cpu.cpp

#include "utils.h"

float square_length(const vector<float>& v) {
    float sum = 0;
    for (size_t i = 0; i < v.size(); i++)
        sum += v[i] * v[i];
    return sum;
}

int main() {
    //srand(time(nullptr));
    const int n = 18;
    vector<float> h_v = get_random_vector(n);

    cout << "v: " << h_v << endl;

    float length = sqrt(square_length(h_v));
    cout << "length: " << length << endl;
    for (int i = 0; i < n; i++)
        h_v[i] /= length;
    length = sqrt(square_length(h_v));
    cout << "length after normalization: " << length << endl;
}

```

1.2 Find Maximum Element

Write in file "max_vec_gpu.cu" a parallel GPU implementation of the following C++ code that finds the maximum **value and index** in a vector. First test with low 'n=18' value but make sure it parallelizes for large 'n=1000000' value and compare the execution time of the GPU implementation with the CPU implementation.

```

max_vec_cpu.cpp

#include "utils.h"

tuple<float, int> max_element(const vector<float>& v) {
    float max_el = -FLT_MAX;
    int max_idx = 0;
    for (size_t i = 0; i < v.size(); i++)
        if (v[i] > max_el) {
            max_el = v[i];
            max_idx = i;
        }
    return tuple<float, int>(max_el, max_idx);
}

int main() {
    //srand(time(nullptr));
    const int n = 18;
    vector<float> h_v = get_random_vector(n);

    cout << "v: " << h_v << endl;

    auto [max_val, max_idx] = max_element(h_v);
    cout << "max element: " << max_val << " at index: " << max_idx << endl;
}

```

1.3 Matrix Vector Product

Write in file "mat_vec_product_gpu.cu" a parallel GPU implementation of the following C++ code that computes the matrix-vector product. First test with low 'rows=18' and 'columns=12' values but make sure it parallelizes

for large 'rows=1000' and 'columns=1000' values and compare the execution time of the GPU implementation with the CPU implementation.

```
mat_vec_product.cpp

#include "utils.h"

int main() {
    //srand(time(nullptr));
    const int rows = 18;
    const int columns = 12;
    vector<float> h_mat = get_random_vector(rows*columns);
    vector<float> h_vec = get_random_vector(columns);
    vector<float> h_result = get_random_vector(rows);

    cout << "Matrix:" << endl;
    print_mat(cout, h_mat, rows, columns);
    cout << "Vector:" << endl;
    print_vec(cout, h_vec);

    for (int r = 0; r < rows; r++) {
        float sum = 0;
        for (int c = 0; c < columns; c++)
            sum += h_mat[r*columns + c] * h_vec[c]; // mat * vec = result
        h_result[r] = sum;
    }

    cout << "Result:" << endl;
    print_vec(cout, h_result);
}
```

1.4 Radix Sort

Some program are much harder to parallelize than others. Sorting is one such example, because a single value can not be processed on its own as its position is dependent on the other values. Write in file "radix_sort_gpu.cu" a parallel GPU implementation of this Python implementation of Radix Sort. First test with low 'n = 15' value but make sure it parallelizes for large 'n=1000000' value and compare the execution time of the GPU implementation with the Python CPU implementation.

2 Submission

Submit your work as "assignment4_solutions.zip" containing the following files:

- "normalize_vec_gpu.cu"
- "max_vec_gpu.cu"
- "mat_vec_product_gpu.cu"
- "radix_sort_gpu.cu"

This zip-file can be created with the following shell command in the 'src' directory:

```
$ make zip
```